



Université de Sherbrooke

Théorie et modèles relationnels

Fondements

TMR_02

Ameni SOUID (ameni.souid@usherbrooke.ca)

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

CoFELI/Scriptorum/TMR_02-Fondements, version 0.1.5.c, en date du 2025-04-14

— version applicable (AS) —

Plan

Introduction	3
1. Présentation	4
2. Théorie relationnelle.....	17
3. Opérateurs relationnels	42

Introduction

Le présent document a pour but de présenter une synthèse de la théorie relationnelle proposée par Edgar F. Codd et développée grâce, notamment, aux contributions de Christopher J. Date, Raymond F. Boyce, Nikos A. Lorentzos et Jeffrey D. Ullman.

La présentation repose sur des bases minimales :

- la logique du premier ordre,
- la théorie des ensembles (dont la théorie des types),
- l'arithmétique entière,
- les langages rationnels.

1. Présentation

La logique et la déduction rationnelle sont les outils privilégiés par la science pour encadrer notre perception du monde, le penser et l'expliquer.

Plus récemment, Floyd, Codd, Hoare, Dijkstra, Gries, Date, Ullman et d'autres ont contribué à rendre effectivement calculable un sous-ensemble significatif des propriétés des systèmes construits sur les bases établies par leurs prédécesseurs.

Ces contributions sont le fondement des systèmes de gestion de bases de données (SGBD) les plus performants développés à ce jour, à savoir les SGBD relationnels (SGBDR).

1.1. Théorie des ensembles

Ensemble

Un ensemble est la réunion d'éléments distincts qui peut être comprise comme un tout [Cantor 1895].

Exemple 1. Ensembles

1. $\{1, 20, 80, 46, 50\}$
2. $\{\text{Paul}, \text{Éliane}, \text{Mohamed}, \text{Sergueï}, \text{Diego}\}$
3. $\{\text{Blanc-Sablon}, \text{Paris}, \text{Tadoussac}, \text{Chandler}, \text{San Diego}\}$
4. $\{\text{IFT187}, \text{IGE687}, \text{IFT159}, \text{IFT677}, \text{STT418}\}$

Tableau 1. Principaux opérateurs ensemblistes

Symbole	Spécification
$\in$	Appartenance
$\subset$	Inclusion
$=$	Égalité
$\cup$	Union
$\cap$	Intersection
$-$	Différence
$\times$	Produit cartésien

Une présentation plus complète est disponibles dans les notes de cours.

Cette association peut être explicite (par voie de déclaration) ou déduite grâce au contexte.

1.1.1. Le couple

Le couple (la paire) est la formation d'une nouvelle entité (ou élément) à partir de deux autres.

Dénotation des valeurs et propriétés

(x; y)

pour x et y des expressions dénotant des valeurs appropriées.

Opérateurs usuels

@1(x; y) = x

l'élément gauche du couple;

@2(x; y) = y

l'élément droit du couple.

1.2. Logique des propositions

Proposition

Une proposition est une assertion ayant une valeur de vérité (vrai ou faux) indépendamment du temps, du lieu ou de la personne qui le prononce.

Tableau 2. Opérateurs de la logique des propositions

Symbole	Spécification
$\neg$	Négation (non)
$\wedge$	Conjonction (et)
$\vee$	Disjonction (ou)
$\Rightarrow$	Implication (si...alors)
$\Leftrightarrow$	Équivalence (si et seulement si)
$\perp$	Faux

Exemple 2. Propositions

- La note de 80 équivaut à la côte A.
- L'activité IFT187 est préalable à l'activité IGE487.
- Éliane habite à Blanc-Sablon.
- Éliane n'est pas inscrite à IFT187.
- Éliane est inscrite à au moins quatre activités.

1.3. Logique des prédicats

La logique des prédicats généralise la logique des propositions. Elle s'intéresse à la formalisation de la sémantique des propositions.

Prédicat

Un prédicat est une expression dont la valeur de vérité dépend des valeurs des variables dont elle est composée.

Nous utiliserons les opérateurs suivants pour les expressions logiques

(adaptation libre de <https://fr.wikipedia.org/wiki/Portail:Logique>, consulté le 2024-04-07).

Exemple 3. Prédicats

- La personne étudiante e est inscrite à l'activité a .
- L'activité a est sous la responsabilité du département d .
- L'activité a est enseignée par la personne p .
- L'activité a est préalable à l'activité b .

Symbole	Spécification
$\neg A$	Négation de A. Non A.
$A \wedge B$	Conjonction. A Et B.
$A \vee B$	Disjonction (inclusive). A Ou B.
$A \Rightarrow B$	Implication. Si A alors B.
$A \Leftrightarrow B$	Équivalence. A est équivalent à B ; on dit aussi : A si et seulement si B.
$\Gamma \vdash A$	Déduction. De l'ensemble de formules Γ on déduit A.
$M \Vdash A$	Réalisabilité. M réalise A, on dit aussi que M « force » A.
$M \models A$	Modélisation. M est un modèle de A ; on dit aussi A est vraie dans M.
$\vdash A$	Théorème. Notion syntaxique.
$\models A$	Tautologie. Notion sémantique.
$(\forall v) A$	Pour chaque v, A est vrai
$(\exists v) A$	Il existe un v où A est vrai

1.4. Relation mathématique

En mathématique, une relation entre deux ensembles est un sous-ensemble du produit cartésien de ces deux ensembles.

Exemple

Soit les deux ensembles suivants :

$Nom = \{Paul, \text{Éliane}, Mohamed, Sergueï, Diego, Eurydice\}$

$Ville = \{Blanc-Sablon, Paris, Tadoussac, Chandler, San Diego, Sherbrooke\}$

Le produit cartésien $Nom \times Ville$ produit 36 couples de valeurs (chaque nom avec chaque ville). Les ensembles suivants sont des relations entre Nom et Ville :

1. $\{(Paul, Paris), (\text{Éliane}, Blanc-Sablon), (Mohamed, Tadoussac), (Sergueï, Chandler)\}$
2. $\{(Paul, Blanc-Sablon), (Mohamed, Chandler)\}$
3. $\{\}$

Les ensembles suivants ne sont pas des relations entre Nom et Ville :

4. $\{(Arthur, Blanc-Sablon), (Mohamed, Montréal)\}$

Selon les définitions, Arthur n'est pas un Nom, Montréal n'est pas une ville.

5. $\{(Paris, Paul)\}$

Selon les définitions, Paris est une Ville, Paul est un Nom, mais l'ordre couple n'est pas respecté.

6. $\{(Paul, Blanc-Sablon), (Paul, Blanc-Sablon)\}$

Deux valeurs identiques ne peuvent apparaître dans un même ensemble.

Notation

Une relation peut également être représentée sous forme tabulaire, ainsi la relation 1 peut être représentée comme suit

Paul	Paris
Éliane	Blanc-Sablon
Mohamed	Tadoussac
Sergeï	Chandler

2. Théorie relationnelle

2.1. Définition informelle

Représentations de la relation informatique

D'abord, voici la représentation usuelle, simple et pratique d'une relation informatique :

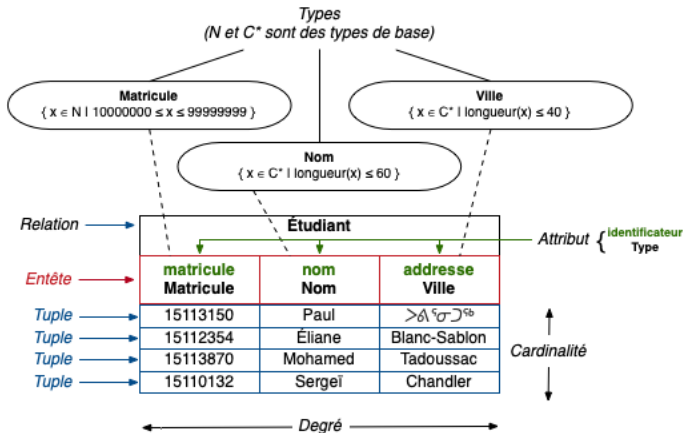
Tableau 3. Étudiant

matricule	nom	ville
15113150	Paul	>Δ ^ς σ ^ς ᵇ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

2.2. Définition formelle

Voici une seconde représentation, plus élaborée et plus précise :

Étudiant {*matricule* : *Matricule*, *nom* : *Nom*, *adresse* : *Ville*}



Relation et prédicat

La relation informatique permet de **structurer**, d'**organiser**, les données.

La relation nous permettra bientôt de définir différents **opérateurs** pour **interroger**, **manipuler** et **traiter** ces données.

Toutefois, pour les **interpréter**, il faut associer un **prédicat** à la relation afin de permettre de déterminer la proposition exacte correspondant à chaque tuple.

Voir l'exemple ci-après.

Si le prédicat suivant est associé à la relation *Étudiant*

«L'étudiant identifié par le matricule "matricule" s'appelle "nom" et est né à "adresse"»,

alors la proposition correspondant au tuple

[matricule:15113150, nom:'Paul', adresse:'Paris']

est

«L'étudiant identifié par le matricule 15113150 s'appelle Paul et est né à Paris».

Alors que, si c'est le prédicat

«L'étudiant identifié par le matricule "matricule" est en stage à "adresse" sous la direction de "nom"»,

la proposition correspondante est plutôt

«L'étudiant identifié par le matricule 15113150 est en stage à Paris sous la direction de Paul».

2.3. Type

Un **type** définit l'ensemble des valeurs qu'une donnée peut prendre et permet de préciser quelles valeurs sont considérées comme valides pour cette donnée.

Il sert donc à **encadrer** les valeurs possibles et à éviter certaines valeurs incohérentes.

Dans le cadre de la théorie minimaliste de typage utilisée dans le présent document, un type est soit un type de base, soit un sous-type.

- Un type de base (appelé aussi type racine ou domaine) est un ensemble fini de valeurs propres (c'est-à-dire que les valeurs d'un type de base n'appartiennent à aucun autre type de base). Une valeur est donc un élément d'un type de base.
- Un sous-type (appelé aussi type dérivé) est un sous-ensemble d'un type, sous-ensemble déterminé par une contrainte explicite (qui restreint les valeurs acceptées dans le sous-ensemble).

Exemples

- Un type de base prédéfini
 - Rel : *Entier*
 - SQL : *Integer*
- Une définition de type de base
 - Rel : *type Enseigne enum (coeur, carreau, trèfle, pique);*
 - SQL : *create type Enseigne as enum ('coeur','carreau','trèfle','pique');*
- Une définition de sous-type
 - Rel : *type Note : Entier | (0 ≤ valeur ≤ 100)*
 - SQL : *create domain Note as Integer check (value between 0 and 100) ;*

2.4. Attribut

Un attribut correspond à une dimension de l'observation, à un « aspect » d'un fait.

Un attribut est un couple défini par :

- un **identifiant** ;
- un **type**.

Il est noté :

$a : D$

où :

- a est l'identifiant de l'attribut ;
- D est son type.

2.5. Tuple

Un tuple, en tant qu'objet de la théorie relationnelle, est une «composition» d'attributs.

En regard de la théorie des types, il s'agit d'un constructeur (de types non scalaires) applicable à un ensemble d'attributs.

Soit n un entier naturel,
 soit n identifiants **distincts** dénotés $a_i \mid 1 \leq i \leq n$,
 soit n types (pas nécessairement distincts) dénotés $D_i \mid 1 \leq i \leq n$,
 soit n valeurs (pas nécessairement distinctes) dénotées $v_i \mid (a_i \in D_i) \wedge (1 \leq i \leq n)$
 alors
 un tuple t est défini comme suit :

$$t \triangleq (\{a_i:D_i \mid 1 \leq i \leq n\}; \{(a_i,v_i) \mid 1 \leq i \leq n\})$$

ou, de façon équivalente, en extension :

$$t \triangleq (\{a_1:D_1, ..., a_i:D_i, ... , a_n:D_n\}; \{(a_1,v_1), ..., (a_i,v_i), ... (a_n,v_n)\})$$

avec

$$\forall i: 1 \leq i \leq \text{deg}(t) \Rightarrow \text{val}(t,a_i) \in \text{def}(t,a_i)$$

où

$$\text{deg}(t) = n$$

$$\text{id}(t) = \{a_i \mid 1 \leq i \leq n\}$$

$$\text{def}(t) = \{a_i:D_i \mid 1 \leq i \leq n\}$$

$$\text{val}(t) = \{(a_i, v_i) \mid 1 \leq i \leq n\}$$

$$\text{ANTE } a \in \text{id}(t):$$

$$\text{def}(t, a) = D \mid a:D \in \text{def}(t)$$

$$\text{ANTE } a \in \text{id}(t):$$

$$\text{val}(t, a) = v \mid (a, v) \in \text{val}(t)$$

degré de t

ensemble des identifiants d'attributs de t

entête de t

valeur de t

type de l'attribut a_i de t

valeur de l'attribut a_i de t

Exemple

[matricule:15113150, nom:'Paul', note:80]

Soit n le nombre d'attributs.

Dans notre exemple : $n = 3$.

- a_i : identifiants des attributs
{matricule, nom, note}
- D_i : types des attributs
{Entier, Texte, Note}
- v_i : valeurs associées
{15113150, 'Paul', 80}

Un tuple t est défini par :

$$t \triangleq (\{a_i:D_i \mid 1 \leq i \leq n\}; \{(a_i,v_i) \mid 1 \leq i \leq n\})$$

La première partie décrit les **attributs et leurs types** :

$\{matricule:Entier, nom:Texte, note:Note\}$

La deuxième partie associe les **attributs à leurs valeurs** :

$\{(matricule,15113150), (nom,'Paul'), (note,80)\}$

Ainsi :

$t = (\{matricule:Entier, nom:Texte, note:Note\}; \{(matricule,15113150), (nom,'Paul'), (note,80)\})$

2.6. Relation

Une relation, en tant qu'objet de la théorie relationnelle, est un ensemble de tuples.

En regard de la théorie des types, il s'agit d'un constructeur (de types non scalaires) applicable à un ensemble de tuples de même type.

Une relation est la représentation d'un prédicat logique qui correspond à un ensemble d'observations de même « nature ».

Soit a_i des identifiants distincts,
soit D_j des types,
soit t_j des tuples alors
une relation R est définie comme suit :

$$R \triangleq (\{a_1:D_1, \dots, a_i:D_i, \dots, a_n:D_n\}; \{t_1, \dots, t_k, \dots, t_m\})$$

avec

$$\forall 1 \leq k \leq \text{card}(R) : \text{def}(R) = \text{def}(t_k)$$

$$t_1 \triangleq (\{\text{matricule:Entier, nom:Texte, ville:Texte}\}; \{(\text{matricule}, 15113150), (\text{nom}, 'Paul'), (\text{ville}, 'Paris')\})$$

$$t_2 \triangleq (\{\text{matricule:Entier, nom:Texte, ville:Texte}\}; \{(\text{matricule}, 15113870), (\text{nom}, 'Mohamed'), (\text{ville}, 'Tadoussac')\})$$

$t_3 \triangleq (\{matricule:Entier, nom:Texte, ville:Texte\} ; \{(matricule, 15114200), (nom, 'Éliane'), (ville, 'Sherbrooke')\})$

$Étudiant \triangleq (\{matricule:Entier, nom:Texte, ville:Texte\} ; \{t_1, t_2, t_3\})$

où

$deg(R) = n$

le degré de R

$card(R) = m$

la cardinalité de R

$id(R) = \{a_1, ..., a_i, ..., a_n\}$

l'ensemble des identifiants d'attributs de R

$val(R) = \{t_1, ..., t_k, ..., t_m\}$

la valeur de R

$def(R) = \{a_1:D_1, ..., a_i:D_i, ..., a_n:D_n\}$

l'entête de R

$def(R, a_i) = D_i$

le type de l'attribut a_i de R

Illustration

Construction de la représentation compacte

Quatre tuples (ayant le même entête)

matricule: Matricule	nom: Nom	adresse: Ville	t1
matricule: 15113150	nom: Paul	adresse: >Δ ^ς σ ^ς ᵇ	
matricule: Matricule	nom: Nom	adresse: Ville	t2
matricule: 15112354	nom: Éliane	adresse: Blanc-Sablon	
matricule: Matricule	nom: Nom	adresse: Ville	t3
matricule: 15113870	nom: Mohamed	adresse: Tadoussac	
matricule: Matricule	nom: Nom	adresse: Ville	t4
matricule: 15110132	nom: Sergeï	adresse: Chandler	

Une relation comprenant quatre tuples

matricule: Matricule	nom: Nom	adresse: Ville	
matricule: Matricule	nom: Nom	adresse: Ville	t1
matricule: 15113150	nom: Paul	adresse: >Δ ^ς σ ^ς ᵇ	
matricule: Matricule	nom: Nom	adresse: Ville	t2
matricule: 15112354	nom: Éliane	adresse: Blanc-Sablon	
matricule: Matricule	nom: Nom	adresse: Ville	t3
matricule: 15113870	nom: Mohamed	adresse: Tadoussac	
matricule: Matricule	nom: Nom	adresse: Ville	t4
matricule: 15110132	nom: Sergeï	adresse: Chandler	

La représentation compacte usuelle de cette même relation

matricule: Matricule	nom: Nom	adresse: Ville
15113150	Paul	>Δ ^ς σ ^ς ᵇ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Prédicat et relation

Un prédicat peut être défini de deux façons et, corolairement, une relation aussi :

- par énumération (l'ensemble de tous les énoncés vrais et eux seuls);
- par compréhension (la caractérisation nécessaire et suffisante des relations entre les variables).

Pour le prédicat :

« *La personne p habite dans la ville v* »

- Par **énumération** :

$\{(Paul, Paris), (Mohamed, Tadoussac), (Éliane, Sherbrooke)\}$

- Par **compréhension** :

$\{(p,v) \mid p \text{ habite dans la ville } v\}$

2.7. Base de données

Une base de données, en tant qu'objet de la théorie relationnelle, est une « composition » de relations.

En regard de la théorie des types, il s'agit d'un constructeur (de types non scalaires) applicable à un ensemble de relations.

Soit v_i des identifiants distincts,
soit D_i des types de relation,
soit r_i des (valeurs de) relations alors
une base B est définie comme suit :

$$B \triangleq (\{v_1:D_1, \dots, v_i:D_i, \dots, v_n:D_n\}; \{r_1, \dots, r_i, \dots, r_n\})$$

avec

$$\forall 1 \leq i \leq \text{card}(B) : \text{def}(B, v_i) = \text{def}(r_i)$$

avec

$deg(B) = n$

$id(B) = \{v_1, ..., v_i, ..., v_n\}$

$val(B) = \{r_1, ..., r_i, ..., r_n\}$

$def(B) = \{v_1:D_1, ..., v_i:D_i, ..., v_n:D_n\}$

$def(B, a_i) = D_i$

degré de B

ensemble des identifiants de B

valeur de B

entête de B

type de a_i de B

Exemple

Considérons une base de données universitaire composée de trois relations :

- *Étudiant* ;
- *Cours* ;
- *Inscription*.

Les identifiants de relations sont :

$v_1 = \textit{Étudiant}$

$v_2 = \textit{Cours}$

$v_3 = \textit{Inscription}$

Les types de relations sont :

$D_1 = \{\textit{matricule:Entier, nom:Texte, ville:Texte}\}$

$D_2 = \{\textit{sigle:Texte, titre:Texte}\}$

$$D_3 = \{matricule:Entier, sigle:Texte, note:Note\}$$

Les valeurs des relations sont :

$$r_1 \triangleq (\{matricule:Entier, nom:Texte, ville:Texte\} ; \{[matricule:15113150, nom:'Paul', ville:'Paris'], [matricule:15113870, nom:'Mohamed', ville:'Tadoussac']\})$$

$$r_2 \triangleq (\{sigle:Texte, titre:Texte\} ; \{[sigle:'IFT187', titre:'Éléments de bases de données'], [sigle:'IFT159', titre:'Analyse et programmation']\})$$

$$r_3 \triangleq (\{matricule:Entier, sigle:Texte, note:Note\} ; \{[matricule:15113150, sigle:'IFT187', note:80], [matricule:15113870, sigle:'IFT187', note:90]\})$$

La base de données peut alors être représentée par :

$$B \triangleq (\{v_1:D_1, v_2:D_2, v_3:D_3\} ; \{r_1, r_2, r_3\})$$

3. Opérateurs relationnels

L'algèbre relationnelle est souvent présentée à l'aide de six opérateurs relationnels primaires :

- **renommage**, $R \rho a:b$, la relation comprenant tous les tuples formés à partir d'un tuple de R dont l'attribut de nom a est remplacé par un attribut de nom b de même valeur, et rien d'autre ;
- **restriction**, $R \sigma c$, la relation comprenant tous les tuples de R satisfaisant la condition c , et rien d'autre ;
- **projection**, $R \pi x$, la relation comprenant tous les tuples formés à partir d'un tuple de R dont seuls les attributs dont le nom est parmi x ont été conservés, et rien d'autre ;

- **jointure**, $R \bowtie S$, la relation comprenant tous les tuples formés des attributs d'un tuple de R et de ceux d'un tuple de S dont les attributs de même nom sont de même valeur, et rien d'autre ;
- **union**, $R \cup S$, la relation comprenant tous les tuples de R et tous les tuples de S , et rien d'autre ;
- **différence**, $R - S$, la relation comprenant tous les tuples de R qui ne sont pas dans S , et rien d'autre.

3.1. Opérateurs primaires

3.1.1. Renommage

R ρ **A:C**

A	B
a1	b1
a2	b2
a3	b3

ρ **A:C** =

C	B
a1	b1
a2	b2
a3	b3

Soit R une relation,
soit a et b deux identifiants alors

$R \rho a:b \equiv$

$R \text{ RENAME } \{a \text{ AS } b\} \equiv$

$\text{select } a_1, \dots, a \text{ as } b, \dots, a_n \text{ from } R \equiv$

$\text{ANTE } a \in \text{id}(R) \wedge b \notin \text{id}(R) :$

$\text{SOIT } E := \text{def}(R, a) ; Z := \text{def}(R) - \{a:E\} \cup \{b:E\} :$

$(Z ; \{(Z ; \text{val}(t) - \{(a, \text{val}(t, a))\} \cup \{(b, \text{val}(t, a))\}) \mid t \in \text{val}(R)\})$

Corolaires

- $\text{card}(R) = \text{card}(R \rho a:b)$
- $\forall t \in \text{val}(R) : (Z ; \text{val}(t) - \{(a, \text{val}(t, a))\} \cup \{(b, \text{val}(t, a))\}) \in R \rho a:b$
- $a \notin \text{id}(R \rho a:b) \wedge b \in \text{id}(R \rho a:b)$
- $R = ((R \rho a:b) \rho b:a)$

3.1.2. Projection

$R \pi \{A, C\}$

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

Soit w une expression ayant pour valeur une liste d'identifiants d'attribut $\{w_1, \dots, w_n\}$
alors

$R \pi w \equiv$

$R \{w_1, \dots, w_n\} \equiv$

$\text{select } w_1, \dots, w_n \text{ from } R \equiv$

$ANTE\ w \subseteq id(R) :$

$SOIT\ Z := \{a:D \mid a \in id(R) \wedge a \in w\} :$

$(Z; \{(Z; \{(a,v) \mid (a,v) \in val(t) \wedge a \in w\}) \mid t \in val(R)\})$

L'opération de projection (applicable tant aux tuples qu'aux relations) n'est bien définie que si tous les identifiants d'attributs sont définis dans l'entête de la relation.

3.1.3. Restriction

R σ cond

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

Soit R une relation,
soit c , une condition (expression booléenne),
soit $id(c)$, l'ensemble des identifiants d'attribut utilisés par c alors

$R \sigma c \equiv$

$R \text{ WHERE } c \equiv$

$\text{select } * \text{ from } R \text{ where } c \equiv$

$ANTE \ id(c) \subseteq id(R) :$

$(def(R); \{t \mid t \in val(R) \wedge c(t)\})$

L'opération de restriction n'est bien définie que si tous les identifiants d'attributs de la condition sont définis dans l'entête de la relation.

3.1.4. Jointure

$R \bowtie S$

A	B
a1	b1
a2	b1
a3	b3
a4	b4

 $\bowtie$

B	C
b1	c1
b2	c2
b3	c3
b3	c4

 $=$

A	B	C
a1	b1	c1
a2	b1	c1
a3	b3	c3
a3	b3	c4

Soit R et S deux relations alors

$R \bowtie S \equiv$

$R \text{ JOIN } S \equiv$

$\text{select } * \text{ from } R \text{ natural join } S \equiv$

$SOIT X := id(R) \cap id(S) :$

$ANTE \forall a \in X : a:D \in def(R) \wedge a:D \in def(S) :$

$SOIT Z := def(R) \cup def(S) :$

$(Z; \{(Z; val(t1) \cup val(t2)) \mid$

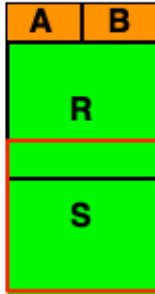
$t1 \in val(R) \wedge t2 \in val(S) \wedge (\forall a \in X : (a,v) \in val(t1) \wedge (a,v) \in val(t2))\})$

Note

- La jointure naturelle n'est pas commutative en SQL en raison de l'ordonnancement des attributs dans un tuple, puisque le tuple y est défini comme une liste ordonnée d'attributs et non comme un ensemble d'attributs.

3.1.5. Union

$R \cup S$



Soit R et S deux relations alors

$R \cup S \equiv$

$R \text{ UNION } S \equiv$

$\text{select } * \text{ from } R \text{ union select } * \text{ from } S \equiv$

$\text{ANTE } \text{def}(R) = \text{def}(S) :$

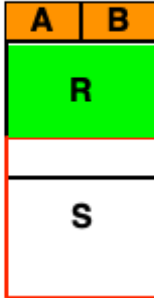
$(\text{def}(R); \text{val}(R) \cup \text{val}(S))$

Note

- L'antécédent exige l'égalité stricte des définitions des relations R et S .
- Il est fréquent de relaxer cette règle, notamment pour prendre en compte la compatibilité opératoire entre deux types dérivés d'un même type de base.
- Attendu une fonction de compatibilité opératoire *CompaOp*, l'antécédent est alors modifié comme suit: $\text{ANTE } \text{CompaOp}(\text{def}(R), \text{def}(S))$.

3.1.6. Différence

R – S



Soit R et S deux relations alors

$R - S \equiv$

$R \text{ MINUS } S \equiv$

$\text{select } * \text{ from } R \text{ except select } * \text{ from } S \equiv$

$ANTE \text{ def}(R) = \text{def}(S) :$

$(\text{def}(R); \text{val}(R) - \text{val}(S))$

.Note * Voir la note de la sous-section relative à l'union.

3.1.7. Synthèse des opérateurs relationnels primaires

Restriction

$R \sigma_{\text{cond}}$

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

Projection

$R \pi\{A, C\}$

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

Jointure

$R \bowtie S$

A	B	B	C
a1	b1	b1	c1
a2	b1	b2	c2
a3	b3	b3	c3
a4	b4	b3	c4

Différence

$R - S$

A	B
R	
S	

Union

$R \cup S$

A	B
R	
S	

Renommage

$R \rho_{A:C}$

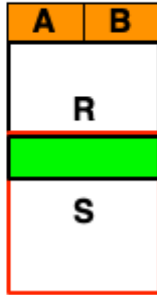
A	B	C	B
a1	b1	a1	b1
a2	b2	a2	b2
a3	b3	a3	b3

3.2. Opérateurs usuels

Afin d'augmenter l'expressivité de l'algèbre relationnelle, il est possible de définir d'autres opérateurs en terme des opérateurs relationnels primaires. En voici quelques-uns.

3.2.1. Intersection

$$\mathbf{R} \cap \mathbf{S} = \mathbf{R} - (\mathbf{R} - \mathbf{S}) = \mathbf{R} \bowtie \mathbf{S}$$



Soit R et S deux relations :

$$R \cap S \equiv$$

$$R \text{ INTERSECT } S \equiv$$

$$\text{select } * \text{ from } R \text{ intersect select } * \text{ from } S \equiv$$

$$\text{ANTE } \text{def}(R) = \text{def}(S) :$$

$$R \bowtie S$$

Notes

- L'antécédent garantit que $R \bowtie S = R - (R - S)$.
- Voir la note de la sous-section relative à l'union.

3.2.2. Produit (cartésien)

$$\mathbf{R} \times \mathbf{S} = \mathbf{R} \bowtie \mathbf{S}$$

A		B			A	B
x		a			x	a
y		b			x	b
		c			x	c
					y	a
					y	b
					y	c

Soit R et S deux relations alors

$R \times S \equiv$

$R \text{ TIMES } S \equiv$

$\text{select } * \text{ from } R \text{ cross join } S \equiv$

$\text{ANTE } id(R) \cap id(S) = \emptyset :$

$R \bowtie S$

Notes

- L'antécédent garantit que $R \bowtie S = R \times S$.
- Voir la note de la sous-section relative à l'union.
- Le produit cartésien n'est pas commutatif en SQL.

3.2.3. Semi-jointure (*matching*)

$$R \bowtie S = (R \bowtie S) \pi R$$

A	B		B	C		A	B
a1	b1	×	b1	c1	=	a1	b1
a2	b1		b2	c2		a2	b1
a3	b3		b3	c3		a3	b3
a4	b4		b3	c4			

Soit R et S deux relations alors

$$R \bowtie S \equiv$$

$$R \text{ MATCHING } S \equiv$$

select $r_1, \dots, r_n$ from R natural join $S \equiv$

$$(R \bowtie S) \pi id(R)$$

3.2.4. Semi-différence (*not matching*)

$$R \ltimes S = R - (R \bowtie S)$$

A	B	⋈	B	C	=	A	B
a1	b1		b1	c1		a4	b4
a2	b1		b2	c2			
a3	b3		b3	c3			
a4	b4		b3	c4			

Soit R et S deux relations alors

$$R \ltimes S \equiv$$

$$R \text{ NOT MATCHING } S \equiv$$

$\text{select } * \text{ from } R \text{ except select } r_1, \dots, r_n \text{ from } R \text{ natural join } S \equiv$

$$R - (R \bowtie S)$$

3.2.5. Extension

$$R \xi \{C := f(A, B)\} = (R \bowtie F) \rho f':C$$

A	B	$\xi \{C := f(A, B)\} =$	A	B	C
a1	b1		a1	b1	f(a1,b1)
a2	b1		a2	b1	f(a2,b1)
a3	b3		a3	b3	f(a3,b3)
a4	b4		a4	b4	f(a4,b4)

Définition

Une relation F correspond à une fonction $f(p_1:T_1, \dots, p_n:T_n)$ de type T lorsque

$$\text{def}(F) = \{p_1:T_1, \dots, p_n:T_n, f:T\} \wedge F = (F \sigma f = f(p_1, \dots, p_n))$$

3.2.5.1. Solution utilisant les seuls opérateurs relationnels primaires

Soit R une relation,

soit F la relation correspondant à la fonction $f(p_1:T_1, \dots, p_n:T_n)$ de type T alors

$R \bowtie \{c := f(p_1, \dots, p_n)\} \equiv$

$R \text{ EXTEND } \{c := f(p_1, \dots, p_n)\} \equiv$

$\text{select } R.*, f(p_1, \dots, p_n) \text{ as } c \text{ from } R \equiv$

$ANTE$

$c \notin id(R) \wedge$

$def(R) \supseteq def(F) - \{f:T\} \wedge$

$R \pi \{p_1, \dots, p_n\} \subseteq F \pi \{p_1, \dots, p_n\} :$

$R \bowtie (F \rho f:c)$

Notes

- L'antécédent signifie simplement que
 - (1) l'identifiant c ne dénote pas déjà un attribut de R ,
 - (2) chaque paramètre de F correspond à un attribut de R ,
 - (3) tout tuple de R à son correspondant dans F .
- La construction de relations en lieu et place des fonctions est un élément important dans la démonstration de la complétude de l'algèbre relationnelle.
- Le symbole utilisé pour l'extension varie beaucoup d'un auteur à l'autre. Certains utilisent α . D'autres présentant l'opération comme une extension de la projection et utilisent conséquemment le symbole π .

Exercice

- En pratique, plusieurs affectations peuvent être spécifiées dans la portée de l'extension, elles sont réputées pouvoir être réalisées concurremment. + En conséquence,

$R \xi \{r_1 := \text{exp}_1 ; r_2 := \text{exp}_2\}$

pourrait ne pas être équivalent à

$(R \xi \{r_1 := \text{exp}_1\}) \xi \{r_2 := \text{exp}_2\}$

en particulier si exp_2 fait appel à l'attribut r_1 .

- Qu'en est-il en **SQL**? Par exemple, quel est le résultat de

```
select 5 as x, x as y
from (values (1, 2)) as A (x, y) ;
```

3.2.5.2. Solution utilisant le modèle

Soit R une relation,

soit $f(p_1:T_1, \dots, p_n:T_n)$ une fonction de type T alors

$R \xi \{c := f(p_1, \dots, p_n)\} \equiv$

$R \text{ EXTEND } \{c := f(p_1, \dots, p_n)\} \equiv$

$\text{select } *, f(p_1, \dots, p_n) \text{ as } c \text{ from } R \equiv$

ANTE

$c \notin \text{id}(R) \wedge$

$\{p_1, \dots, p_n\} \subseteq \text{id}(R) :$

SOIT

$Z := \text{def}(R) \cup \{c:T\} :$

$(Z; \{(Z; \text{val}(t) \cup \{(c, f(t.p_1, \dots, t.p_n))\}) \mid t \in \text{val}(R)\})$

Produit le 2026-09-06 14:43:05 UTC



Université de Sherbrooke